

Installationsguide: MCP-server (mq-mcp) på macOS

Denna guide beskriver hur du driftsätter och konfigurerar mq-mcp, en referensserver baserad på Model Context Protocol (MCP). Som Senior AI-infrastrukturtekniker fokuserar vi här på att eliminera "bespoke glue" – den skräddarsydda och bräckliga kod som ofta krävs för att koppla samman LLM-applikationer med interna verktyg.

1. Introduktion och Arkitekturöversikt

MCP standardiserar hur AI-modeller interagerar med din infrastruktur. Genom att implementera denna server löser vi tre kritiska utmaningar:

1. **Kontextfragmentering:** Istället för att skicka med hela chatthistoriken tillåter MCP oss att hämta precis rätt kontext via resurser.
2. **Integrationsskuld:** Vi slipper bygga unika API-kopplingar för varje ny agent; MCP skapar en enhetlig yta för verktyg, resurser och prompts.
3. **Komponerbarhet:** Vi kan dela upp domäner (t.ex. beräkningar och externa API:er) i separata servrar som alla ser identiska ut för klienten.

Den mentala arkitekturmodellen

- **Host:** LLM-applikationen (t.ex. Claude Desktop eller VS Code) som initierar anslutningen.
- **Client:** En komponent i hosten som upprätthåller en 1:1-session med servern via JSON-RPC.
- **Server (mq-mcp):** Exponerar Tools (beräkningar), Resources (statisk data) och Prompts (instruktionsmallar).

2. Systemkrav och Förberedelser

För en robust utvecklingsmiljö på macOS krävs följande verktyg:**Checklista för förberedelser:**

- **Homebrew:** Den primära pakethanteraren för macOS.
- **Git:** För versionshantering.
- **Node.js:** Krävs för att köra MCP Inspector (byggd i TypeScript), även om själva servern är i Python.
- **uv:** Modern och extremt snabb pakethanterare för Python (ersätter pip och venv).

3. Miljökonfiguration med uv och Node

Vi väljer uv eftersom det ger omedelbar feedback och isolerade miljöer utan krångel, vilket är avgörande vid iterativ utveckling av verktygsbeskrivningar.

Installation av kärnverktyg:

```
# Installera uv för Python-ekosystemet
brew install uv

# Installera Node.js (krävs för MCP Inspector)
brew install node
```

4. Projektinitiering och Struktur

Vi skapar en ren projektstruktur där `server.py` agerar som serversnypunkt.

```
# Initiera projektet
uv init mq-mcp
cd mq-mcp

# Lägg till beroenden (mcp-cli krävs för dev-mode)
uv add "mcp[cli]" requests
```

Rekommenderad filstruktur:

- `server.py`: All serverlogik och registrering av verktyg.
- `pyproject.toml`: Hanteras automatiskt av `uv`.
- `.vscode/mcp.json`: För lokal integration i VS Code.

5. Implementation av MCP-server med FastMCP

Vid implementationen är docstrings inte bara dokumentation; de är de instruktioner som LLM:en använder för att förstå när ett verktyg ska anropas. "Plain language wins tool selection" är vår gyllene regel. Skapa `server.py` med följande innehåll:

```
import random
import requests
from mcp.server.fastmcp import FastMCP

# Initiera servern med ett tydligt namn
mcp = FastMCP("mq-mcp")

# --- Tools ---
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two integers and return the sum. Use this for all basic
    math additions."""
    # Senior-tips: Validering direkt i verktyget förhindrar
    # felaktiga LLM-anrop
    if not isinstance(a, int) or not isinstance(b, int):
        raise ValueError("Båda parametrar (a och b) måste vara
        heltal.")
    return a + b

@mcp.tool()
def joke(topic: str | None = None) -> str:
    """Fetch a random Chuck Norris joke. Optional topic refines
    the joke."""
    try:
        if topic:
            res =
requests.get("https://api.chucknorris.io/jokes/search",
params={"query": topic}, timeout=5)
            data = res.json()
```

```

        results = data.get("result", [])
        if results:
            return random.choice(results).get("value")

    res =
requests.get("https://api.chucknorris.io/jokes/random", timeout=5)
    return res.json().get("value", "Kunde inte hitta ett skämt
just nu.")
    except Exception as e:
        return f"Ett fel uppstod vid hämtning av skämt: {str(e)}"

# --- Resources ---
@mcp.resource("reference://readme")
def readme() -> str:
    """Statisk resurs som ger instruktioner om serverns
förmågor."""
    return (
        "Reference MCP Server\n\n"
        "Verktyg som stöds:\n"
        "- add(a, b): Addera heltal.\n"
        "- joke(topic): Hämta skämt.\n"
        "Använd dessa för att verifiera korrekt verktygsval och
parameteröverföring."
    )

# --- Prompts ---
@mcp.prompt(name="math_add_prompt", description="Recipe for using
add/joke reliably.")
def math_add_prompt() -> str:
    """Returnerar den fullständiga instruktionsmallen från
källmaterialet."""
    return (
        "You are using the Reference MCP server.\n\n"
        "When the user asks to add numbers:\n"
        "1) Call the add tool with integers a and b.\n"
        "2) Confirm inputs before executing.\n"
        "3) Reply with just the numeric result.\n\n"
        "If the user asks for a joke:\n"
        "1) If they provide a topic, call joke with that topic.
Otherwise call joke with no topic.\n"
        "2) Reply with the joke text only."
    )

if __name__ == "__main__":
    mcp.run()

```

6. Testning och Validering med MCP Inspector

Innan vi ansluter till Claude Desktop måste vi verifiera JSON-RPC-handskakningen (initialize -> initialized).

GUI-baserad testning

1. Starta inspektören: `npx @modelcontextprotocol/inspector`
2. I webbläsaren (ofta port 6274), välj transporttyp **STDIO**.
3. Ange kommando: `uv run mcp dev server.py`
4. **Säkerhet:** Inspektören körs via en proxy. Om anslutningen misslyckas, leta efter en säkerhetsnyckel i terminalfönstret, klicka på "Configuration" i GUI:t och klistra in den.

CLI-baserad testning (för CI/CD)

Använd `mcp run` för stabila kontroller:

```
# Kontrollera att verktygen exponeras korrekt
npx @modelcontextprotocol/inspector --cli uv run mcp run server.py
--method tools/list
```

Förväntade testresultat:

Testfall, Förväntat resultat (JSON-RPC)

`tools/list`, En lista med `add` och `joke` inklusive deras beskrivningar.

`"add(a=18, b=2)"`, Ett content-objekt av typen `text` med värdet `20`.

`reference://readme`, Den statiska textsträngen definierad under `Resources`.

7. Integration med Claude Desktop och VS Code

För att host-applikationer ska kunna spåna din serverprocess krävs konfiguration via `mcp.json`.

Claude Desktop (macOS)

Konfigurationsfilen finns på: `~/Library/Application`

`Support/Claude/claude_desktop_config.json`!CAUTION **Absoluta sökvägar:** Claude

Desktop startar servern i en separat miljö. Du **måste** använda absoluta sökvägar till både `uv` och `server.py` för att undvika tysta startfel.

```
{
  "mcpServers": {
    "mq-mcp": {
      "command": "/opt/homebrew/bin/uv",
      "args": [
        "--directory", "/absolut/sökväg/till/mq-mcp",
        "run",
        "mcp",
        "run",
        "server.py"
      ]
    }
  }
}
```

Starta om Claude Desktop helt efter att du sparat filen.

VS Code (GitHub Copilot Agent Mode)

Skapa .vscode/mcp.json i projektets rot:

```
{
  "servers": {
    "mq-mcp": {
      "command": "uv",
      "args": ["run", "mcp", "run", "server.py"]
    }
  }
}
```

8. Best Practices och Felsökning

!!IMPORTANT Stdio-begränsning: Vid användning av stdio-transport (standard för lokala processer) får servern aldrig skriva till stdout via print(). Detta korrumpierar JSON-RPC-strömmen och bryter anslutningen. Använd logging till stderr för debugging.

Vanliga problem vid driftsättning:

- **Handskakningsfel:** Om servern inte svarar på initialize, kontrollera att inga bakgrundsprocesser av servern redan körs och ockuperar resurser.
- **Beskrivningsmissar:** Om LLM:en väljer fel verktyg, gör docstringen mer specifik. Istället för "Lägg till", använd "Addera två heltal och returnera summan".
- **Sökvägar:** Verifiera att uv finns i din PATH genom att köra which uv i terminalen och använd resultatet i din konfiguration.

Framåtblick: Från Stdio till Streamable HTTP

För produktion och fjärrdriftsättning bör man migrera från stdio till **Streamable HTTP**. Detta förenklar hantering av proxies, autentisering och gör servern tillgänglig för flera klienter samtidigt, vilket är nästa steg i din MCP-resa.